Recruit researchers Join for free Login

Preprint PDF Available

Estimating Numerical Solutions to Laplace's Equation and their Error to the Analytical via Finite Difference Methods

March 2020
DOI:10.13140/RG.2.2.29660.62087
License · CC BY 4.0

Authors:

Marc Nairn
University of Glasgow

Preprints and early-stage research may not have been peer reviewed yet.

Download file PDF Download citation Copy link

References (5) Figures (1)

Abstract and Figures

This work presents close estimations to solutions of Laplace's equation using two distinct finite difference methods: the Gauss-Seidel and Jacobi methods. Periodic boundary conditions are applied to their algorithms so as to facilitate the simulations of larger environments, as well as a comparison of this implementation instead of a "multi-grid" setup. The accuracy of these methods is quantified by their error, measured as the average difference between the numerical and analytical solutions over the simulated grid as iterations increase. In both cases the resulting error is similar and exponentially decreasing, reaching around 0.2% for 10 5 iterations. The Jacobi method shows a quicker average convergence of all its grid-points. However, it is not very significant and there is reason to believe the Gauss-Seidel might show better individual grid-point convergence if this was instead measured due to its spread in time indices in its coded algorithm. The Gauss-Seidel method is chosen to produce the numerical plots to the solutions as it shows better CPU performance, besting its counterpart by margins of up to an hour when iterations increase to 10 8. These results may be improved by instead using a mixture of both methods and accelerating relaxation by implementing more sophisticated numerical schemes such as the Successive Over Relaxation method.

Visualisation of the Gauss-Seid…

Figures - uploaded by Marc Nairn Author content
Content may be subject to copyright.

Discover the world's research

- 25+ million members
- 160+ million publication pages
- 2.3+ billic citations

Join for free

Public Full-text 1

Content uploaded by Marc Nairn Author content
Content may be subject to copyright.

Estimating Numerical Solutions to Laplace's Equation and their Error to the Analytical via Finite Difference Methods

2329589n

January 28, 2021

Abstract

This work presents close estimations to solutions of Laplace's equation using two distinct finite difference methods: the Gauss-Seidel and Jacobi methods. Periodic boundary conditions are applied to their algorithms so as to facilitate the simulations of larger environments, as well as a comparison of this implementation instead of a "multi-grid" setup. The accuracy of these methods is quantified by their error, measured as the average difference between the numerical and analytical solutions over the simulated grid as iterations increase. In both cases the resulting error is similar and exponentially decreasing, reaching around 0.2% for 10^5 iterations. The Jacobi method shows a quicker average convergence of all its grid-points. However, it is not very significant and there is reason to believe the Gauss-Seidel might show better individual grid-point convergence if this was instead measured due to its spread in time indices in its coded algorithm. The Gauss-Seidel method is chosen to produce the numerical plots to the solutions as it shows better CPU performance, besting its counterpart by margins of up to an hour when iterations increase to 10^8. These results may be improved by instead using a mixture of both methods and accelerating relaxation by implementing more sophisticated numerical schemes such as the Successive Over Relaxation method.

1 Introduction

Solutions to many of the problems in physics rely heavily on the development of accurate numerical methods, as analytical solutions are hard to come by when the environments get increasingly complex. These such methods result in numerical solutions which are close to the ones that could be obtained via analytical manipulation. If the methods are consistent, the numerical solution will agree with the exact one in the iteration or, alternatively, discretisation step limit. Most of these techniques assume the convergence of the solution can be fitted to a polynomial function of the iteration steps. All the numerical schemes used provide solutions to arbitrary problems as iterations increase. The major drawbacks of these techniques are the time required for the simulations (the CPU time itself is a function of the number of iterations) and the available memory (as this one will also decrease the longer the simulation is ran). The accuracy of a certain method is determined by comparing their numerical solutions to the analytical ones to provide a measure of a relative error (rather by plotting the average difference of all the points between the numerical and analytical solutions), their convergence is also measured by averaging the change in the solution from the current and the previous iteration and the CPU times as functions of iterations are also measured.

This work presents a study on numerical schemes used to determine solutions to Laplace's equation, as well as their accuracy and CPU performance when set against each other.

Laplace's equation, presented here in its two dimensional form, allows the potential V to be written as:

$$\nabla^2 V = \frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = 0, \tag{1}$$

Solving Eqn. 1 with specified boundary conditions for a given system will allow V to be determined which will in turn allow the **E**-field of the system to be determined.

§2 will further develop the reasoning behind using numerical methods to solve partial differential equations and introduce the ideas behind finite difference methods (FDMs), as well as the algorithms behind the FDMs used, the Gauss-Seidel and Jacobi FDMs. In §3 the quality of the results produced by the FDMs is quantified, checking for accuracy of the results, the computing time required for them, and how fast these converge to a stable solution. §4 will recap all the work done, pointing out the flaws of each the Gauss-Seidel and Jacobi methods (and why the Gauss-Seidel

has been chosen to be the better choice) as well as presenting some possible improvements, such as implementing the Successive Over Relaxation method instead so as to increase convergence speed.

2 Numerical Solutions to Laplace's Equation

2.1 Discretising Laplace's Equation via Finite Difference Methods

Derivatives, such as Eqn. 1 may be closely approximated via the use of finite difference methods (FDMs). The derivative for an analytical function $f(x)$ can be approximated as:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{2}$$

where h is assumed to be a small finite value, instead of infinitesimally small as in the classical definition of a derivative. Now, considering the Taylor expansion:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{3} f'''(x) + \dots \tag{3}$$

Which becomes

$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h). \tag{4}$$

when rearranging and grouping all higher order terms under the error term $O(h)$.

By considering the Taylor expansion of the difference of the function values $f(x+h)$ and $f(x-h)$, the central difference equation follows,

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2) \approx \frac{f(x+h) - f(x-h)}{2h} \tag{5}$$

This derivation allows a derivative to be approximated with an error of order h^2, a factor of h times better than the standard, one-sided difference method (Eqn. 4). Second order differentials may also be approximated by Taylor expanding first order derivatives as in Eqn. 3 and, following the same reasoning as Eqn. 5, resulting in,

$$f''(x) \approx \frac{f'(x) - f'(x-h)}{h} = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}. \tag{6}$$

To compute the potential given from Eqn. 1, the equation must be discretized. A constant grid spacing in the x and y directions may be set. For this purpose a rectangular body whose boundaries are conducting surfaces of size (n $\times$ m) may be created. The grid spacing in the x-direction is denoted by h_x, and by h_y in the y-direction; but since the spacing is set to be constant: $h_x = h_y = h$.

So Eqn. 1 may be reformulated considering this setup and, similar to Eqn. 6, by approximating the partial differentials individually and assuming h to be small.

$$\frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} \approx \frac{V(x+h,y) - 2V(x,y) + V(x-h,y)}{h^2} + \frac{V(x,y+h) - 2V(x,y) + V(x,y-h)}{h^2} = 0$$

Recall the Taylor expansion higher order terms are grouped under the error term $O(h^2)$:
Now, gathering terms and solving for central potential point $V(x,y)$:

$$V(x,y) = \frac{1}{4}\left[V(x+h,y) + V(x-h,y) + V(x,y+h) + V(x,y-h)\right]$$

or, using the indexed notation k, l:

$$V(x,y) = \frac{1}{4}\left(V_{k+1,l} + V_{k-1,l} + V_{k,l+1} + V_{k,l-1}\right) \tag{7}$$

2.2 Implementing Finite Difference Methods in the code

In order to apply the finite difference formula (Eqn. 7) derived in §2.1, efficient and accurate algorithms must be devised with the aim of achieving suitable numerical solutions which will agree with the analytical ones. For this project, two distinct, well known, algorithms have been used to generate numerical solutions to the setups from the problems:the Gauss-Seidel (GS) and the Jacobi. Both are fairly easy and straightforward applications of the finite difference formula to an arbitrarily sized grid, iteratively applying Eqn. 7 onto all points on the grid.

2.2.1 Gauss-Seidel method

The GS method evaluates the (x, y) term's potential in "real-time" by taking the most recent values of its four neighbouring points. This "real-time" evaluation results in different time indices, q, corresponding to the iteration number, used for the point's neighbours.The finite difference methods in the code are applied iteratively, line by line, column by column, from the 0th index to the last. Therefore, on iteration $q + 1$ point $V(x, y)$ will have 2 neighbours already computed at q, $(V(x - 1, y), V(x, y - 1))$, while the other two, $(V(x + 1, y), V(x, y + 1))$, are uncomputed at $q + 1$ and thus their most recent computed value at q is taken, resulting in Algorithm 1. The GS results in CPU time being optimised as it stores the least amount of potential values as possible, fitting all of those it needs into a single potential matrix.

The GS algorithm can be visualised in Fig. 1, where a simplified 4×4 grid with PBCs is computed according to Algorithm 1.

Result: Apply Gauss-Seidel FDM on every point of an n×m grid
Initialise voltage matrix:
→ set boundary voltage conditions on initially empty n×m matrix
repeat
  for *index*$_x$ **in** $0..n$ **do**
    for *index*$_y$ **in** $0..m$ **do**
      $V^{q+1}_{x,y} = \frac{1}{4}\left(V^{q+1}_{x-1,y} + V^{q}_{x+1,y} + V^{q+1}_{x,y-1} + V^{q}_{x,y+1}\right)$
    end
  end
for ***N*** *iterations*;

Algorithm 1: General Finite Difference Gauss-Seidel algorithm for *N* iterations

(a) (c)

(b)

Figure 1: Visualisation of the Gauss-Seidel algorithm with periodic boundaries filling in a simplified 4×4 grid by computing the Current grid point averaging over four neighbouring grid points in "real-time". Half of the points (labelled "computed") used to compute the average belong to the current iteration $q + 1$ (the Gauss-Seidel algorithm has been applied to them in $q + 1$), while the other half left (labelled uncomputed) are the results from the previous iteration's (q) calculation; as grid fills out (1b, 1c) there are less points belonging to the q-th iteration. Note the use of Periodic Boundary Conditions allows to mirror the grid when grid-edge is reached in the iteration (1b, 1c).

2.2.2 Jacobi method

Similar to the GS algorithm, The Jacobi method also employs the previous iteration of the code to compute the current iteration's potential values, $V(x, y)$. The main difference between the two comes in the time indices, q, used in their finite difference formula. As opposed to doing it in real time, like its counterpart, the Jacobi method saves two full grids corresponding to the current $q+1$ and previous q iterations. For point $V(x, y)$ in iteration $q+1$, its new potential value is calculated by applying Eqn. 7 on the point at the previous iteration. At the end of each iteration, the $q+1$ "current" matrix is pushed into the q "old" matrix, so that the $q+1$ can be computed again by using the old voltage matrix. Algorithm 2 highlights its code implementation.

This method requires two full grid-sized matrices to be stored at all times, so longer running times for the code may be expected.

Result: Iteratively apply Jacobi-FDM on every point of an n×m grid.
Initialise empty current potential, $V^{q+1}_{n,m}$, and old potential, $V^{q}_{n,m}$, matrices and set initial boundary conditions on them
repeat
  for $index_x$ **in** $0..n$ **do**
    for $index_y$ **in** $0..m$ **do**
      $V^{q+1}_{x,y} = \frac{1}{4}(V^{q}_{x-1,y} + V^{q}_{x+1,y} + V^{q}_{x,y-1} + V^{q}_{x,y+1})$
    end
  end
  $V^{q} \leftarrow V^{q+1}$
for ***N*** *iterations*;

Algorithm 2: General Finite Difference Jacobi algorithm for *N* iterations

Figure 2: Visualisation of the Jacobi algorithm applied on a simplified 4×4 grid with Periodic Boundary Conditions. Two copies of the grid are stored: 2a shows the current iteration q and highlights the current point $V(x, y)^{q+1}$ in the algorithm, 2b shows the finite difference equation being applied onto the previous iterations point $V(x, y)^{q}$ by averaging over its four nearest neighbours and whose result will be stored in $V(x, y)^{q+1}$

3 Benchmarking

Another one of the aims of this project is to, as well as being able to reproduce and simulate any arbitrary set-up's V-field, quantify which of the methods used is the most efficient and accurate. For this, both CPU time and time until convergence need to be checked; or, more accurately, CPU time as a function of iterations and convergence as a function of iterations as these increase.

For the former, the code is run a few times for an increasing number of iterations (from $1 \cdot 10^3$ to $5 \cdot 10^5$) for each of the different methods used, resulting in a total of seven distinct data sets, see below in Fig. 3. For this section, the cylinder between parallel plates setup from Problem 2 was used as an example to benchmark each method.

Computation time over number of iterations for problem 2

100

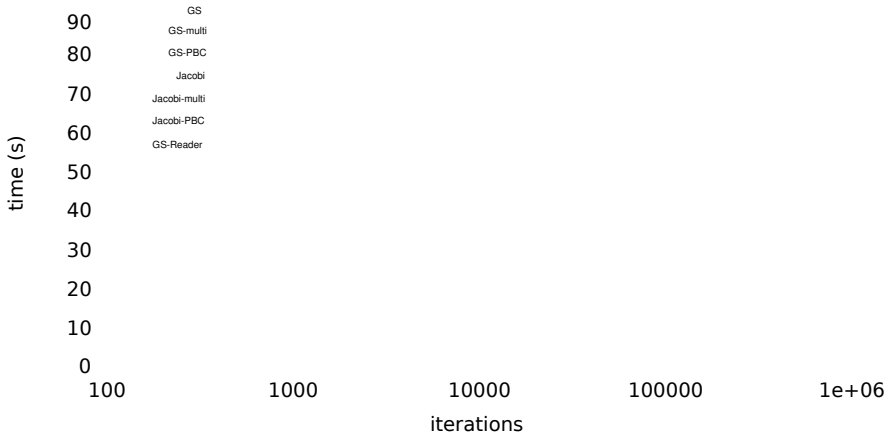

Figure 3: CPU time for each of the methods used to obtain the scalar potential field for the cylinder between two parallel plates setup, for a 100×100 grid. Plot highlights computing time is directly proportional to both number of iterations, N, and grid-size ("multi-grids" require triple the grid area to be computed, and take three times as longer to compute). Periodic boundary conditions result in slightly longer computing time as an extra check is set throughout the finite difference iterations for the edge grid-points. Integrating the "Reader" into the code, results in an overall lengthier compilation (about triple the time) as two full data files need to be read in iteratively before producing the potential data set. The Jacobi method tends to be marginally slower than the Gauss-Seidel method in this iteration range, though it shows an exponential nature which will result in this difference to increase at at higher iterations

Fig. 3 confirms what could have been predicted: the running time of the code is proportional to both the number of iterations and the size of the computed grid, and hinting at an exponential relationship. Periodic boundary conditions slightly increase the run-time of the code, compared to standard, static, boundary conditions, as they require the gridpoint's location to be checked iteratively and acted upon should it be out of range. Implementing the reader into the code makes this one about three times slower, compared to inputting the grid potential parameters in the code. This could be because it requires two .txt files, each with as many data points as points in the grid, to be read before even starting the main finite difference algorithm.

Also in Fig. 3, a very minor difference in computing time between the Jacobi and the Gauss-Seidel FDMs can be observed, with the Jacobi's time per iteration always slighly longer than its counterpart. This is not too unexpected as the Jacobi algorithm uses and stores two matrices, side by side, in order to compute each point's potential value, thus resulting in more storage space being used. However, since all the points show an increasing exponential curve, the CPU time difference between these two methods may also considerably increase should iterations or grid-size (or both) go beyond this range.

To more clearly visualise this, the two methods may be isolated and plotted in the most simple static boundary conditions setup. To their plots, a best fit line is extrapolated from the data, with $a + cx^b$: $\{a, b, c \in \mathbb{R}\}$ as the initial guess of its functional form. These values are approximated within a confidence interval by *Mathematica*, yielding the functional equations to the best-fit lines of the Jacobi and Gauss-Seidel finite difference methods:

$$f(x)_{\text{Gauss-Seidel}} \approx 0.0174522 + 0.0000685869x^{0.995641} \tag{8}$$

$$f(x)_{\text{Jacobi}} \approx 0.0532008 + 0.0000632373x^{1.02646} \tag{9}$$

The values of a and c can effectively be disregarded, as only the exponent, b, of the functions is of interest. In the low N case (N$\sim$ 10^5) Eqns. 8 and 9 are $\leq$2 seconds apart; the exponential terms are small enough, and so is the difference between them, that no discernible difference can be seen before N is increased way beyond this value, Fig. 4c. In this high iteration range, a small increase in N will result in a sizeable difference in CPU times for both methods.

Citations (0) References (5)

A Numerical Solution of the 2D Laplace's Equation for the Estimation of Electric Potential Distribution

Article Full-text available

Dec 2018

Tersoo Atsue · Emmanuel Vezua Tikyaa

View Show abstract

Estimating the error between analytical and numerical finite difference solutions of laplace equation

Conference Paper Full-text available

Nov 2017

Alcyone Silva · Leonardo R A X Menezes

View

Finite Difference Method for the Solution of Laplace Equation

Article

Ambar K. Mitra

View

Solving the Heat, Laplace and Wave equations using finite difference methods

Jan 2012

A Peirce

A. Peirce, Solving the Heat, Laplace and Wave equations using finite difference methods, Partial Differential Equations, Department of Mathematics, University of British Columbia, 2012.

Jan 2020

A Hameed · S Gallacher · A Gosnay

A. Hameed, S. Gallacher, A. Gosnay, M. Nairn An Assessment of Numerical Methods in Electrostatics, P3 Theoretical Physics Group Report, School of Physics and Astronomy, University of Glasgow, 2020.

Recommended publications Discover more

Sponsored content

Scotland's most sustainable university

June 2023

The University of Glasgow is Scotland's most sustainable university according to the Times Higher Education Impact Rankings 2023. Ranked 13th in the world overall and 2nd in the UK, the University of Glasgow's was the first University in Europe to announce it would divest from fossil fuels and...

View post

Sponsored content

Adam Smith 300 Year Anniversary – Global Reading Group Events

February 2023

We invite you to join the University of Glasgow's Adam Smith Global Reading Group. The group is an exclusive opportunity to delve into Adam Smith's seminal book, The Wealth of Nations, as part of the Adam Smith's Tercentenary celebrations. A chance to join as a global community to read, examine...

View post

Sponsored content

Adam Smith 300 Year Anniversary – Global Reading Group Events

February 2023

We invite you to join the University of Glasgow's Adam Smith Global Reading Group. The group is an exclusive opportunity to delve into Adam Smith's seminal book, The Wealth of Nations, as part of the Adam Smith's Tercentenary celebrations. A chance to join as a global community to read, examine...

View post

Sponsored content

The future with quantum

November 2022

The Centre for Quantum Technology is advancing the UK's lead in quantum technology while training the country's future experts. Researchers at the University of Glasgow are making sure that quantum technology is having a significant impact across climate change, healthcare, computing, and...

View post

Preprint Full-text available

An Assessment of Numerical Methods in Electrostatics

April 2020

Marc Nairn · Arman Hameed · S Gallacher · A Gosnay

Electrostatic boundaries can modify the electric field with the inclusion of objects with different potentials. A detailed plot is generated, analytically and numerically, to illustrate the potential and electric fields within these different environments. Using the two distinct finite difference methods, Gauss-Seidel and Jacobi, numerical solutions for potential and electric field are computed, ... [Show full abstract]

View full-text

Technical Report Full-text available

An Assessment of Numerical Methods in Electrostatics

March 2020

Arman Hameed

Electrostatic boundaries can modify the electric field with the inclusion of objects with different potentials. A detailed plot is generated, analytically and numerically, to illustrate the potential and electric fields within these different environments. Using the two distinct finite difference methods, Gauss-Seidel and Jacobi, numerical solutions for potential and electric field are computed, ... [Show full abstract]

View full-text

Technical Report Full-text available

Finding Potential Solutions for Various Electrostatic Systems

March 2020

Arman Hameed

Electrostatic boundaries can modify the electrostatic potential field with the inclusion of objects with different potentials. A detailed plot is generated analytically and numerically; to illustrate the potential fields within these different environments. Using the two distinct finite difference methods, Gauss-Seidel and Jacobi, numerical solutions for the potential field is computed, which can ... [Show full abstract]

View full-text

Article Full-text available

Decomposed element-free Galerkin method compared with finite-difference method for elastic wave prop...

May 2009 · Geophysics

Masafumi Katou · Toshi Matsuoka · Hitoshi Mikada · [...] · Yuzuru Ashida

The decomposed element-free Galerkin (DEFG) method is a modified scheme to resolve shortcomings of memory use in element-free Galerkin (EFG) methods. DEFG solves elastic wave equation problems by alternately updating the stress-strain relations and the equations of motion as in the staggered-grid finite-difference (FD) method. DEFG requires at most twice the memory space, a size comparable to ... [Show full abstract]

View full-text

Download on the App Store

GET IT ON Google Play

Company

About us
News
Careers

Support

Help Center

Business solutions

Advertising
Recruiting

© 2008-2023 ResearchGate GmbH. All rights reserved.

Terms · Privacy · Copyright · Imprint